LLM Post-Training: A Deep Dive into Reasoning Large Language Models

Group Members:

Taimoor Amir (21L-6268)
Zahra Hussain (21L-5615)
Ahmad Sohaib (21L-6274)
Saim Maroof (21L-5611)

Introduction

- LLMs are powerful but can give wrong or confusing answers (hallucination).
- Pre-training is not enough — models need refining.
- Post-training improves reasoning, accuracy, and human alignment.
- **Fine-tuning**, **Reinforcement Learning**, and **Scaling** are key.

Future: Better alignment, adaptation, reasoning

Why Post Training?

LLMs often:

- Lose logical flow
- Misunderstand user intentions
- Struggle with complex reasoning

Post-training fixes reliability, bias, and context understanding.
Without it, LLMs are not fully safe for real-world use

Reinforced LLMs: Overview

- Enhances model reasoning and safety post-pretraining.
- Key stages: Supervised Fine-Tuning (SFT), Reward Modeling, RL Fine-Tuning.
- Ensures models produce aligned, useful outputs.

Supervised Fine-Tuning (SFT)

- Fine-tunes base model on curated datasets.
- Instills format, style, and baseline task understanding.
- Acts as the “base policy” for RL steps

Reward Modeling: Types & Roles

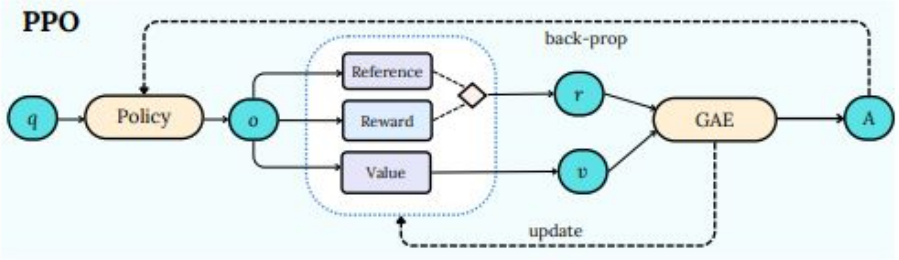
- Trains a model to assign scores based on human preferences.
- Reward types: Explicit vs Implicit, Outcome vs Process.
- RM guides RL updates by acting as a “judge”.

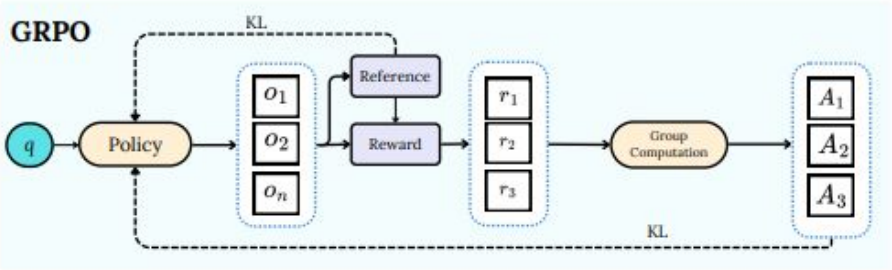
Reward Model Training Process

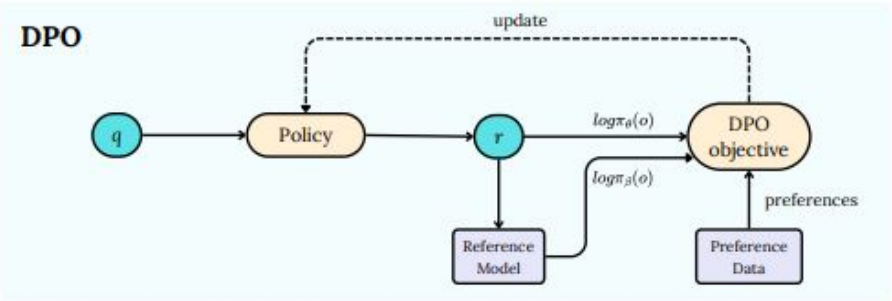
- Collect prompts + multiple responses.
- Human rank or compare outputs.
- Train RM to assign scalar reward scores to preferred responses.

Policy optimization

- **ORPO:** Directly optimizing a policy from pairwise human preferences.
- **PPO:** Stable RL with clipping + KL penalty.
- **RLHF**: Combines SFT, reward modeling, and PPO .
- **TRPO:** Update the policy only within a safe zone called a "trust region."
- **DPO:** Training language models directly from preference data.
- **OREO:** Enhance LLMs’ multi-step reasoning by optimizing the soft Bellman equation.
- **GRPO:** Samples a group of outputs from the model and estimates rewards based on comparing.

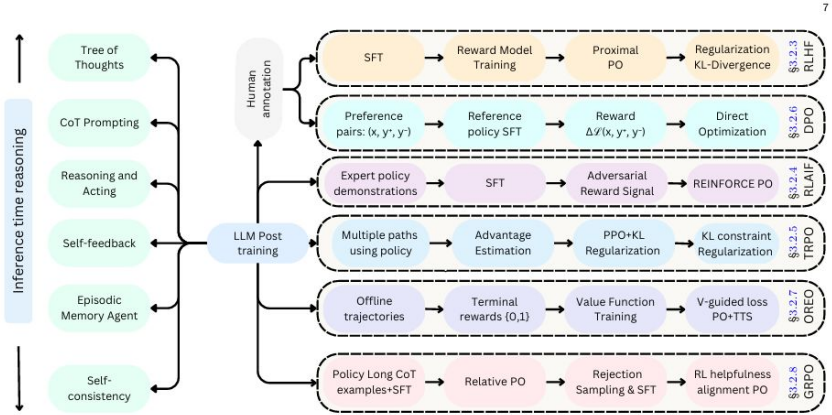

PPO
back-prop
q
Policy
o
Reference
Reward
Value
r
v
GAE
A
update


GRPO
KL
q
Policy
o_1
o_2
o_n
Reference
Reward
r_1
r_2
r_3
Group Computation
A_1
A_2
A_3
KL


DPO
update
q
Policy
r
$log\pi_\theta(o)$
DPO objective
$log\pi_\beta(o)$
preferences
Reference Model
Preference Data

Pure RL Based LLM Refinement

- **DeepSeek-R1-Zero** operates only with RL, meaning it is trained purely through reinforcement learning without any supervised fine-tuning (SFT).
- **DeepSeek-R1** integrates cold-start data and uses a multi-stage pipeline that includes RL, SFT, distillation, and addressing challenges like language mixing and readability.



RL Based LLM Refinement

Cold-Start RL Phase:

- This phase involves gathering a small amount of data to provide a foundation for the model.

Rejection Sampling and Fine-Tuning:

- Once the RL phase stabilizes, **rejection sampling** is used to generate high-quality responses then added to the training dataset for further fine-tuning.

Reasoning-Oriented RL:

- This approach uses **GRPO** to generate multiple outputs from the model and then evaluate each output based on specific criteria.

Second RL Stage for Human Alignment:

- In this stage, the model is further refined using reward signals based on **human preferences.**

Distillation for Smaller Models:

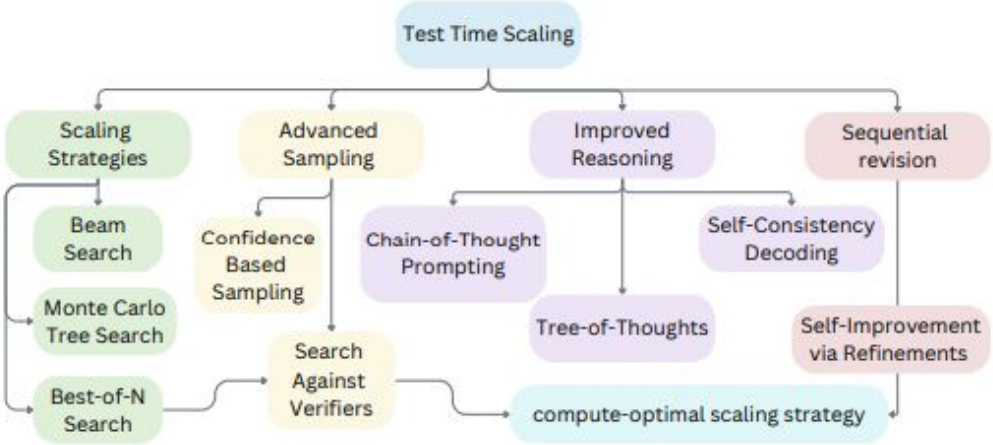
- After the large model (DeepSeek-R1) is fully trained, **distillation** is used to transfer its capabilities to smaller models.

Supervised Fine-tuning in LLMS

1. **Instruction Fine-Tuning:** Train on instruction-response pairs.
2. **Dialogue (Multi-turn) Fine-Tuning:** Focus on multi-turn conversations.
3. **Chain-of-Thought (CoT) Reasoning:** Model learns step-by-step reasoning
4. **Domain Specific :** Tailored for specialized domains
5. **Distillation-Based Fine-Tuning:** Large teacher models guide smaller student models.
6. **Preference and Alignment SFT:** Teaches models user values: safe vs. toxic outputs.
7. **Efficient Fine-Tuning (PEFT):** By only updating a small part of the model.

5. Test Time Scaling

- Improve LLM **performance** and **efficiency** without changing the model's architecture
- Helps LLMs **generalise** better, stay **accurate**



Smaller models with test-time compute can outperform much larger models in certain scenarios.

Test Time Scaling

Pretraining vs Test Time Scaling:

- PT: making the model bigger and training it on more data
  - Uses lots of resources
- TTS: smarter computation only when the model is actually answering (inference)
  - Improve accuracy
  - Significant cost saving
- TTS can match models 14x its size!

Test-time compute is not a 1-to-1 replacement for pretraining but, offers a viable alternative in many cases.

Test Time Scaling Methods

1. **Beam Search:**
   - First used in speech recognition, later adopted in machine translation and LLMs.
   - At each step, keeps only the top N most likely partial sequences.
   - Improves chances of finding a correct and coherent solution.

2. **Best-of-N Search (Rejection Sampling)**
   - Generates N full outputs and selects the best based on a chosen criteria
   - Rejection sampling: sample multiple, keep the best.
   - **Easy to implement**, with N being the main tunable parameter.
   - Encourages **diverse outputs** and works well for easier or creative tasks
   - Can suffer from **reward hacking**, instability if N is too large.

Test Time Scaling Methods

3. **Compute-Optimal Scaling (COS):**

    - Dynamically allocates compute during inference
    - Adapts to the difficulty of each prompt
    - 5 difficulty levels: easy to hard
    - For easy: sequential refinement, iteratively improves answer
    - For hard: parallel sampling, beam search

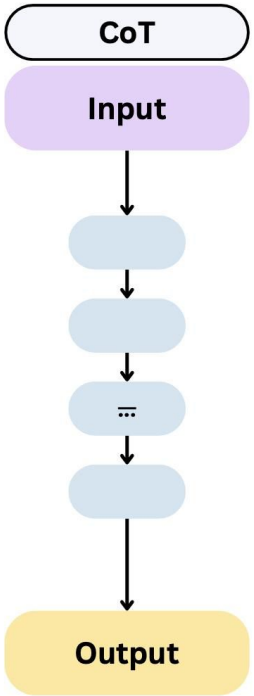
4. **Confidence-Based Sampling:**

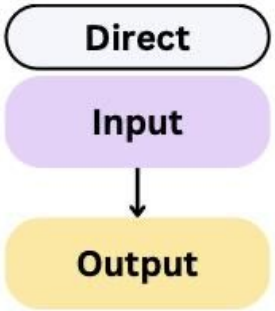
    - Use confidence scores to rank answers
    - Helps allocate more compute to low-confidence answers
    - Avoids wasting resources
    - Model relies on its own assessment, can be wrong

Test Time Scaling Methods

5. **Chain-of-Thought (CoT):**

- "Think out loud" through step-by-step reasoning
- Breaks down tasks into smaller, manageable steps just like humans
- Outperforms direct questioning on arithmetic, logic, math problems.

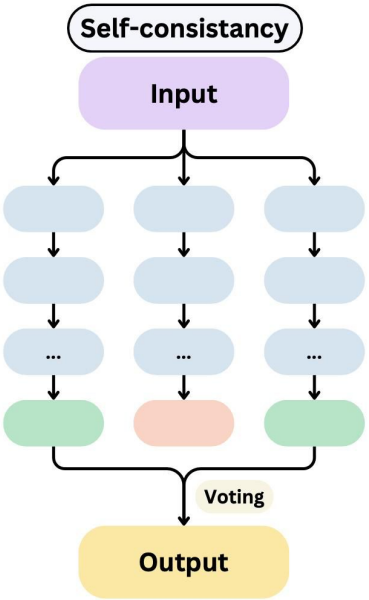




Test Time Scaling Methods

6. Self Decoding Sampling:

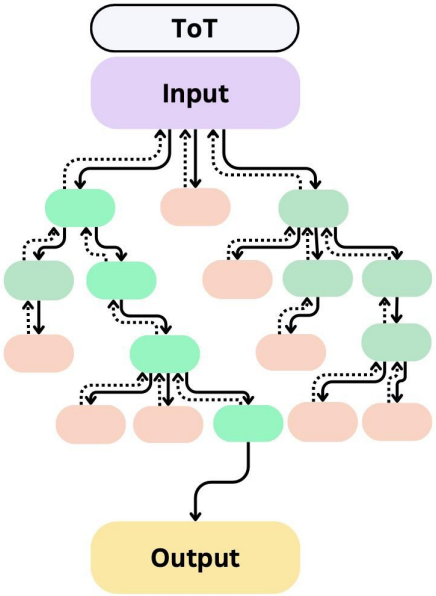
- Sample multiple reasoning paths
- Final answer is selected by majority vote: most frequent answer
- Can be expensive to run
- More diversity, accuracy
- Greatly improves accuracy on complex reasoning tasks



Test Time Scaling Methods

7. Tree-of-Thoughts (ToT):

- Try several different ideas, explore multiple possibilities
- Model can branch out, look ahead, backtrack
- Significantly better reasoning and planning performance
- Effective for complex tasks requiring multi-step reasoning,
  - puzzles, planning, and games, where exploring multiple strategies
- Monte Carlo Search
  - handle large search spaces by sampling intelligently rather than exhaustively

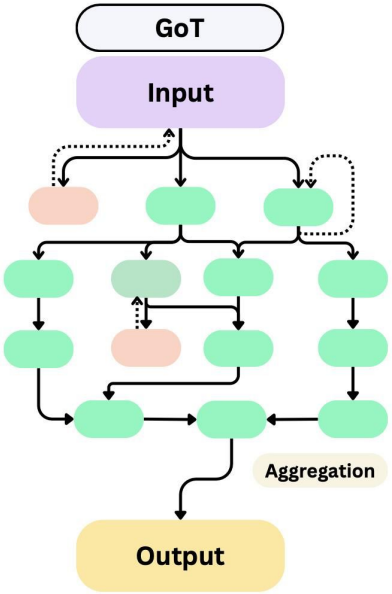


Test Time Scaling Methods

8. Graph of Thoughts (GoT):

- Flexible network, where each thought can connect to any other.
- No strictly hierarchy, can edit previous thoughts
- Three main transformations:
  - **Aggregation**: merging multiple solution paths into a single answer
  - **Refinement**: iteratively improving thoughts by backtracking
  - **Generation**: producing diverse candidate solutions.

- GoT can significantly improve task performance
  - 62% better sorting quality and 31% lower cost compared to ToT



Test Time Scaling Methods

9. Search Against Verifiers:

- Separates the process of generating solutions (proposer) from evaluating them (verifier)
- Verifier checks which answer is most correct
  - student and teacher
- **Outcome Reward Models** (ORM): judge only the final answer
- **Process Reward Models** (PRM): evaluate each step in the reasoning chain

10. Self-Improvement via Refinements:

- Review and revise answers during inference, checking for mistakes, and update
- **Self Refinement**: answer generation, self-critique, refinement, and iteration
- **Self Polish**: makes the problem statement itself clearer and easier to solve
- Improves LLM performance on complex problem-solving
  - mathematical reasoning, code generation

Self-improvement methodologies represent a paradigm shift in LLM optimization, emphasizing active reasoning and internal feedback over static pre-training. By iterating on their own responses, models achieve greater consistency and accuracy across a wide range of applications.

Conclusion

- Post-training is essential to make LLMs useful, safe, and smart.
- Fine-tuning, RL, and Scaling together transform basic LLMs into high-quality assistants.
- Future work will focus on safety, efficiency, and smarter reasoning abilities.

Thank you!